xArm Lite 6 Challenge Instructions

CTC vs PD/PID Under Perturbations

Task-Space via Inverse Kinematics

1 Overview and Objective

1.1 Primary Objective

Implement a real-robot task on the xArm Lite 6 where the task is defined in Cartesian space and joint references are generated using inverse kinematics (IK). Performance must be compared between a model-based controller (Computed Torque Control or CTC) and a baseline joint-space controller (PD or PID). The robustness of both controllers must be evaluated under external perturbations.

1.2 Learning Outcomes

Upon completion, you will demonstrate proficiency in task-space motion planning with IK-based trajectory generation, implementation of weighted resolved-rate IK with singularity handling, model-based versus model-free control comparison, experimental design and fair controller comparison methodology, and quantitative performance analysis and reporting.

2 Reference Code and Conceptual Framework

Critical Requirement

You **must** explicitly reference any code you used from external sources. Your report must include a dedicated section explaining how your implementation corresponds to the components in this reference code.

2.1 Required IK Components from Reference Code

Your implementation must include all of the following components. First, you must implement axis-weighted resolved-rate IK with Z-priority using a weight matrix $W = \mathrm{diag}(1, 1, w_z)$ where $w_z > 1$, which prioritizes vertical (Z-axis) tracking accuracy. Second, you must use the damped least-squares (DLS) pseudo-inverse with formula $J^{\#} = J_w^T(J_w J_w^T + \lambda^2 I)^{-1}$ using small damping λ to avoid suppressing Z authority while preventing singularity issues. Third, nullspace posture stabilization must be included to prevent drift to low-manipulability configurations using the home configuration as a posture reference. Fourth, consistent reference generation is required, producing $q_{des}(t)$, $\dot{q}_{des}(t)$, and $\ddot{q}_{des}(t)$ using the lightweight $\dot{J}$ estimate from the reference code.

Finally, the controller comparison must be fair, with both controllers tracking the same IK-generated references on the same plant with identical perturbations.

3 Controller Selection and Requirements

3.1 Baseline Controller: PD vs PID

Required

Choose your baseline controller based on your previous lab work. Use PD control if your previous labs used proportional-derivative control without integral action. Use PID control if your previous labs included integral action.

3.2 PID-Specific Requirements

If using PID, you must include anti-windup via integral clamping or an equivalent method. You must also justify your integral limits in the report.

3.3 Universal Controller Requirements

Both controllers (CTC and PD/PID) must include actuator saturation handling with documented limits, rate limiting appropriate for safe operation, and emergency stop capability.

4 Task Definition Requirements

Critical Requirement

You must define your own task in Cartesian space meeting ALL of the following criteria:

Your task must include a minimum of 8 distinct waypoints with explicit coordinates (x, y, z) provided in meters within the base frame. The waypoints must form a non-trivial path rather than just a straight line. Additionally, you must define at least two distinct Z levels including a "low layer" height and a "high layer" height with minimum vertical separation of 0.08 m.

Each waypoint must include a dwell period with minimum dwell time of 1.0 second per waypoint. These dwells are used for steady-state tracking evaluation and must be clearly identifiable in data logs. The task must conclude by returning to the home configuration, with the final waypoint returning to the initial position to create a closed-loop trajectory. All transitions between waypoints must be smooth with no instantaneous velocity or acceleration jumps, using appropriate trajectory generation methods such as trapezoidal or quintic splines.

4.1 Documentation Requirements

In your report, provide a table with waypoint number, coordinates (x, y, z) in meters, layer designation (low or high), dwell duration at each waypoint, and transition time to next waypoint.

5 Inverse Kinematics Implementation

5.1 Required IK Method

Required

Implement weighted resolved-rate IK consistent with the reference code, including the following components:

For Z-priority weighting, use

$$W = \mathrm{diag}(1, 1, w_z) \quad \text{with } w_z > 1 \tag{1}$$

$$J_w = WJ \tag{2}$$

with recommended $w_z \in [2.0, 3.5]$. For the DLS pseudo-inverse, implement

$$J^{\#} = J_w^T (J_w J_w^T + \lambda^2 I)^{-1} \tag{3}$$

with recommended damping $\lambda \in [0.01, 0.02]$. Nullspace posture stabilization must use

$$\dot{q}_{null} = (I - J^{\#} J_w)(-k_{null}(q - q_{home})) \tag{4}$$

with recommended $k_{null} \in [1.0, 2.5]$. Finally, task-space error feedback should be implemented as

$$\dot{q}_{des} = J^{\#}(\dot{p}_{des} + k_{task}(p_{des} - p)) + \dot{q}_{null} \tag{5}$$

with recommended $k_{task} \in [10, 20]$.

5.2 Reference Signal Generation

You must generate desired joint positions $q_{des}(t)$, desired joint velocities $\dot{q}_{des}(t)$, and desired joint accelerations $\ddot{q}_{des}(t)$ which are required for CTC. Use the lightweight $\dot{J}$ estimation scheme from the reference code to ensure consistency in generating these signals.

6 Perturbation Requirements

Critical Requirement

Perturbations are MANDATORY. The comparison is invalid without them.

6.1 Perturbation Configuration

Critical requirement: Keep perturbation configuration **identical** across controller runs.

6.2 Implementation Options

If your lab provides a perturbation injector node, use the ROS2 perturbation node as shown in the example command below. You must log all parameters used.

```
ros2 run xarm_perturbations perturbation_injector --ros-args \
  -p output_topic:=/servo_server/delta_twist_cmds \
  -p enabled:=true \
  -p mode:=gaussian \
  -p gauss_std_linear:=0.5 \
  -p gauss_axis:=x
```

If implementing perturbations differently, you must document the exact mechanism (whether force, torque, position offset, etc.), all parameters (magnitude, frequency, direction, timing), verification that perturbations are identical across runs, and the method for synchronizing perturbation injection with the control loop.

6.3 Perturbation Logging

You must log perturbation enable and disable timestamps, perturbation signal values (if available), and all perturbation parameters.

7 Required Experimental Trials

Required

Execute exactly four trials using identical task definition and IK reference generator:

Trial	Controller	Perturbations	Baseline?
1	CTC	No	No
2	PD/PID	No	Yes
3	CTC	Yes	No
4	PD/PID	Yes	Yes

7.1 Trial Execution Requirements

For each trial, you must begin from the same initial configuration (document this configuration explicitly in your report). Use the same IK-generated references for both controllers, execute the complete task including all waypoints plus return to home, log data for the entire duration, and save separate log files for each trial with clear naming.

7.2 Trial Naming Convention

Use consistent naming such as
`trial_ctc_nopert_YYYY-MM-DD_HH-MM-SS`,
`trial_pdpid_nopert_YYYY-MM-DD_HH-MM-SS`,
`trial_ctc_pert_YYYY-MM-DD_HH-MM-SS`, and
`trial_pdpid_pert_YYYY-MM-DD_HH-MM-SS`.

8 Data Logging Requirements

Critical Requirement

All trials must be logged with time-stamped data at the control rate.

8.1 Required Data Fields

For every trial, log the following at each time step. Record time stamps including both absolute time (ROS time or system time) and relative time from trial start. Log joint-space measurements including measured joint positions $q(t) \in \mathbb{R}^6$ and measured joint velocities $\dot{q}(t) \in \mathbb{R}^6$. Record joint-space references including desired positions $q_{des}(t) \in \mathbb{R}^6$, desired velocities $\dot{q}_{des}(t) \in \mathbb{R}^6$, and desired accelerations $\ddot{q}_{des}(t) \in \mathbb{R}^6$ if used by the controller. Log task-space data including measured end-effector position $p(t) \in \mathbb{R}^3$ and desired end-effector position $p_{des}(t) \in \mathbb{R}^3$. Record controller outputs including commanded torques $\tau_{cmd}(t) \in \mathbb{R}^6$ if available, or commanded setpoints (effort, position, or velocity depending on interface), along with saturation events recorded as binary flags for each joint. Finally, log perturbation data including perturbation signals if available, perturbation enable flag, and perturbation parameters logged once at start.

8.2 Logging Method

Use rosbag or your lab's official logging tool. Export to CSV format for analysis. Include a metadata file with trial identifier, controller type, perturbation configuration, controller gains and parameters, and IK parameters.

9 Performance Metrics and Analysis

9.1 Required Metrics

Compute the following metrics for **all four trials**:

9.1.1 Joint-Space Metrics

For each joint $j = 1, \ldots, 6$, compute the Root Mean Square Error (RMSE) as $\text{RMSE}_j = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(q_j[k] - q_{des,j}[k])^2}$, the maximum absolute error as $e_{max,j} = \max_k |q_j[k] - q_{des,j}[k]|$, and the dwell-window mean error computed for each waypoint w as $\bar{e}_{j,w} = \frac{1}{N_w}\sum_{k\in\text{dwell}_w} |q_j[k] - q_{des,j}[k]|$.

9.1.2 Task-Space Metrics

Compute the end-effector position error norm as $e_{EE}(t) = \|p(t) - p_{des}(t)\|_2$. From this, calculate the RMSE of end-effector error as $\text{RMSE}_{EE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_{EE}[k]^2}$ and the maximum end-effector error as $e_{EE,max} = \max_k e_{EE}[k]$. Additionally, compute the mean and max error during each dwell window.

9.1.3 Waypoint Success Rate

Define a success threshold ϵ_{thresh} (for example, 0.005 m). For each waypoint, check if $e_{EE} < \epsilon_{thresh}$ during the final 0.5 seconds of the dwell period. Compute the success rate as Success Rate $= \frac{\text{Number of successful waypoints}}{\text{Total waypoints}} \times 100\%$. You must justify your threshold choice in the report.

10 Required Plots

Generate the following plots for **each trial**:

10.1 Joint Tracking Plots

Generate joint position tracking plots with 6 subplots showing $q_j(t)$ versus $q_{des,j}(t)$ for $j = 1, \ldots, 6$. Clearly label axes with units, include a legend, and highlight dwell windows using shaded regions or similar visual indicators.

10.2 Task-Space Plots

Create end-effector component tracking plots with 3 subplots showing $x(t)$ versus $x_{des}(t)$, $y(t)$ versus $y_{des}(t)$, and $z(t)$ versus $z_{des}(t)$. Mark waypoints clearly on these plots. Generate a 3D path visualization plotting the actual path and desired path in 3D space, marking waypoints and the home position, and including a coordinate frame reference. Additionally, plot the end-effector error norm over time showing $e_{EE}(t) = \|p(t) - p_{des}(t)\|_2$, including a success threshold line and highlighting perturbation periods if applicable.

10.3 Phase Portrait Analysis

Generate phase portraits showing e_j versus $\dot{e}_j$ for a minimum of 3 joints (preferably all 6). These plots show stability and convergence behavior and should annotate the equilibrium point at the origin.

10.4 Comparison Plots

Create controller comparison plots overlaying CTC versus PD/PID for the same perturbation condition. Generate separate plots for no-perturbation and with-perturbation scenarios.

10.5 Summary Comparison Table

Required

Create a comprehensive table comparing all four trials:

Metric	No Perturbations		With Perturbations	
	CTC	PD/PID	CTC	PD/PID
Joint RMSE (avg)				
Joint Max Error (avg)				
EE RMSE (mm)				
EE Max Error (mm)				
Waypoint Success (%)				

11 Code Structure Template

11.1 Imports and Configuration

```
# Cell 0: Configuration
import numpy as np
import rclpy
# ... other imports

# Control parameters
CONTROL_RATE_HZ = 200
DT = 1.0 / CONTROL_RATE_HZ

# Controller selection
USE_PID = True   # Set False for PD only

# Safety limits
TORQUE_LIMIT = 10.0  # Nm (adjust based on your robot specs)
VELOCITY_LIMIT = 2.0  # rad/s
```

11.2 Task Definition

```
# Cell 1: Define task-space waypoints
# TODO: Replace with your own task meeting all requirements

# Example structure (MUST REPLACE):
waypoints = [
    # Low layer (z = 0.15)
    (0.30,  0.10, 0.15),  # waypoint 1
    (0.35,  0.10, 0.15),  # waypoint 2
    (0.35, -0.10, 0.15),  # waypoint 3
    (0.30, -0.10, 0.15),  # waypoint 4

    # High layer (z = 0.25)
    (0.30,  0.10, 0.25),  # waypoint 5
    (0.35,  0.10, 0.25),  # waypoint 6
    (0.35, -0.10, 0.25),  # waypoint 7
    (0.30, -0.10, 0.25),  # waypoint 8

    # Return to waypoint 1 (close the loop)
    (0.30,  0.10, 0.15),  # waypoint 9
]

# Timing parameters
DWELL_SEC = 1.0      # dwell at each waypoint
```

```
SEGMENT_SEC = 2.0     # transition time between waypoints

# TODO: Generate smooth trajectory with dwells
# p_des(t), p_dot_des(t), p_ddot_des(t)
```

11.3 Forward Kinematics and Jacobian

```
# Cell 2: Kinematics utilities
# TODO: Implement or import from reference code

def forward_kinematics(q):
    """
    Compute end-effector position from joint angles.

    Args:
        q: (6,) array of joint angles [rad]

    Returns:
        p: (3,) array of end-effector position [m]
    """
    # TODO: Implement using DH parameters or robot model
    pass

def position_jacobian(q):
    """
    Compute position Jacobian (3x6).

    Args:
        q: (6,) array of joint angles [rad]

    Returns:
        J: (3, 6) position Jacobian matrix
    """
    # TODO: Implement using DH parameters or robot model
    pass
```

11.4 Weighted Resolved-Rate IK

```
# Cell 3: IK reference generator
# TODO: Implement based on reference code

# IK parameters (tune these)
wz = 2.5          # Z-axis weight (> 1.0)
lam = 1.5e-2      # DLS damping
k_task = 14.0     # Task-space error gain
k_null = 1.5      # Nullspace posture gain

q_home = np.array([0, 0, 0, 0, 0, 0])  # Define your home config

def weighted_resolved_rate_ik(ts, p_des, p_dot_des, p_ddot_des):
    """
    Generate joint references using weighted resolved-rate IK.

    Args:
```

```
        ts: (N,) time array
        p_des: (N, 3) desired positions
        p_dot_des: (N, 3) desired velocities
        p_ddot_des: (N, 3) desired accelerations

    Returns:
        q_des: (N, 6) desired joint positions
        qd_des: (N, 6) desired joint velocities
        qdd_des: (N, 6) desired joint accelerations
    """
    N = len(ts)
    q_des = np.zeros((N, 6))
    qd_des = np.zeros((N, 6))
    qdd_des = np.zeros((N, 6))

    # TODO: Implement IK loop
    # - Initialize from q_home
    # - At each step:
    #   * Compute J = position_jacobian(q)
    #   * Apply weighting: W = diag(1, 1, wz)
    #   * Compute DLS pseudo-inverse
    #   * Add nullspace posture term
    #   * Integrate to get q_des[i]
    #   * Estimate J_dot for qdd_des

    return q_des, qd_des, qdd_des
```

11.5 PD/PID Baseline Controller

```
# Cell 4: Baseline PD/PID controller

# Gains (TODO: tune these based on your system)
Kp = np.diag([40, 40, 40, 30, 25, 20])
Kd = np.diag([14, 14, 14, 10,  8,  6])

if USE_PID:
    Ki = np.diag([10, 10, 10,  6,  4,  3])
    i_limit = 0.6  # Anti-windup limit
    e_int = np.zeros(6)

def baseline_controller(q, qd, q_ref, qd_ref, dt):
    """
    PD or PID joint-space tracking controller.

    Args:
        q: (6,) measured joint positions
        qd: (6,) measured joint velocities
        q_ref: (6,) desired joint positions
        qd_ref: (6,) desired joint velocities
        dt: time step

    Returns:
        tau: (6,) commanded torques
    """
    global e_int
```

```
    # Tracking errors
    e = q - q_ref
    e_dot = qd - qd_ref

    if USE_PID:
        # Integral with anti-windup
        e_int = np.clip(e_int + e * dt, -i_limit, i_limit)
        tau = -(Kp @ e + Kd @ e_dot + Ki @ e_int)
    else:
        tau = -(Kp @ e + Kd @ e_dot)

    # Saturation
    tau = np.clip(tau, -TORQUE_LIMIT, TORQUE_LIMIT)

    return tau
```

11.6 CTC Controller

```
# Cell 5: Computed Torque Control

# CTC gains (TODO: tune these)
Kp_ctc = np.diag([80, 80, 80, 60, 50, 40])
Kd_ctc = np.diag([28, 28, 28, 20, 16, 12])

# Robust/sliding term parameters (optional but recommended)
gamma = 2.5
k_robust = 5.5
sign_eps = 2e-3

def smooth_sign(x, eps=1e-3):
    """Smooth approximation of sign function."""
    return np.tanh(x / eps)

def get_robot_dynamics(q, qd):
    """
    Get robot dynamics matrices.

    Args:
        q: (6,) joint positions
        qd: (6,) joint velocities

    Returns:
        M: (6, 6) mass matrix
        Cqd: (6,) Coriolis/centrifugal vector
        G: (6,) gravity vector
        F: (6,) friction vector
    """
    # TODO: Implement using your robot model
    # Can use analytical model or robot_state_publisher
    pass

def ctc_controller(q, qd, q_ref, qd_ref, qdd_ref):
    """
    Computed Torque Control with optional robust term.

    Args:
```

```
        q: (6,) measured joint positions
        qd: (6,) measured joint velocities
        q_ref: (6,) desired joint positions
        qd_ref: (6,) desired joint velocities
        qdd_ref: (6,) desired joint accelerations

    Returns:
        tau: (6,) commanded torques
    """
    # Get dynamics
    M, Cqd, G, F = get_robot_dynamics(q, qd)

    # Tracking errors
    e = q - q_ref
    e_dot = qd - qd_ref

    # Feedback linearization
    v = qdd_ref - (Kp_ctc @ e) - (Kd_ctc @ e_dot)

    # CTC law
    tau_ctc = M @ v + Cqd + G + F

    # Optional robust/sliding term
    S = gamma * e + e_dot
    tau_robust = k_robust * smooth_sign(S, sign_eps)

    tau = tau_ctc + tau_robust

    # Saturation
    tau = np.clip(tau, -TORQUE_LIMIT, TORQUE_LIMIT)

    return tau
```

11.7 Real-Robot Execution

```
# Cell 6: Trial execution framework

def run_trial(controller_name, enable_perturbations=False):
    """
    Execute one trial.

    Args:
        controller_name: "ctc" or "baseline"
        enable_perturbations: bool

    Returns:
        log_data: dict of logged arrays
    """
    # TODO: Implement using your lab's ROS2 interface

    # 1. Initialize ROS2 node
    # 2. Reset robot to home position
    # 3. Start/configure perturbations if enable_perturbations=True
    # 4. Initialize data logging arrays

    # Main control loop
```

```
    rate = CONTROL_RATE_HZ
    t = 0.0
    idx = 0

    while idx < len(ts):
        # Read current state
        q_meas = read_joint_positions()
        qd_meas = read_joint_velocities()

        # Get reference at current time
        q_ref = q_des[idx]
        qd_ref = qd_des[idx]
        qdd_ref = qdd_des[idx]

        # Compute control
        if controller_name == "ctc":
            tau_cmd = ctc_controller(q_meas, qd_meas,
                                     q_ref, qd_ref, qdd_ref)
        else:  # baseline
            tau_cmd = baseline_controller(q_meas, qd_meas,
                                          q_ref, qd_ref, DT)

        # Send command
        send_joint_torques(tau_cmd)

        # Log data
        # log_data["time"].append(t)
        # log_data["q"].append(q_meas)
        # ... log all required fields

        # Compute end-effector position
        p_meas = forward_kinematics(q_meas)
        p_ref = p_des[idx]
        # log_data["p"].append(p_meas)
        # log_data["p_des"].append(p_ref)

        idx += 1
        t += DT
        # rate.sleep()

    # 5. Stop perturbations
    # 6. Save logs to rosbag and CSV

    return log_data

# Execute all four required trials
# trial1 = run_trial("ctc", enable_perturbations=False)
# trial2 = run_trial("baseline", enable_perturbations=False)
# trial3 = run_trial("ctc", enable_perturbations=True)
# trial4 = run_trial("baseline", enable_perturbations=True)
```

11.8 Analysis and Plotting

```
# Cell 7: Post-experiment analysis

import matplotlib.pyplot as plt
```

```python
from mpl_toolkits.mplot3d import Axes3D

def compute_metrics(trial_data):
    """
    Compute all required metrics for one trial.

    Args:
        trial_data: dict with keys 'q', 'q_des', 'p', 'p_des', etc.

    Returns:
        metrics: dict of computed metrics
    """
    metrics = {}

    # Joint-space metrics
    q = np.array(trial_data['q'])
    q_des = np.array(trial_data['q_des'])
    e_joint = q - q_des

    metrics['joint_rmse'] = np.sqrt(np.mean(e_joint**2, axis=0))
    metrics['joint_max_error'] = np.max(np.abs(e_joint), axis=0)

    # TODO: Compute dwell-window metrics

    # Task-space metrics
    p = np.array(trial_data['p'])
    p_des = np.array(trial_data['p_des'])
    e_ee = np.linalg.norm(p - p_des, axis=1)

    metrics['ee_rmse'] = np.sqrt(np.mean(e_ee**2))
    metrics['ee_max_error'] = np.max(e_ee)

    # Waypoint success rate
    # TODO: Implement based on your threshold

    return metrics

def plot_trial(trial_data, title_prefix):
    """Generate all required plots for one trial."""

    # Joint tracking plots
    fig, axes = plt.subplots(3, 2, figsize=(12, 10))
    fig.suptitle(f'{title_prefix}: Joint Tracking')

    for j in range(6):
        ax = axes[j // 2, j % 2]
        ax.plot(trial_data['time'], trial_data['q'][:, j],
                label='Measured')
        ax.plot(trial_data['time'], trial_data['q_des'][:, j],
                '--', label='Desired')
        ax.set_xlabel('Time [s]')
        ax.set_ylabel(f'$q_{j+1}$ [rad]')
        ax.legend()
        ax.grid(True)

    plt.tight_layout()

    # Task-space plots
```

```
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    fig.suptitle(f'{title_prefix}: Task-Space Tracking')

    # X, Y, Z components
    for i, label in enumerate(['X', 'Y', 'Z']):
        if i < 3:
            ax = axes[i // 2, i % 2]
            ax.plot(trial_data['time'], trial_data['p'][:, i],
                    label='Measured')
            ax.plot(trial_data['time'], trial_data['p_des'][:, i],
                    '--', label='Desired')
            ax.set_xlabel('Time [s]')
            ax.set_ylabel(f'{label} [m]')
            ax.legend()
            ax.grid(True)

    # 3D path
    ax = fig.add_subplot(224, projection='3d')
    ax.plot(trial_data['p'][:, 0], trial_data['p'][:, 1],
            trial_data['p'][:, 2], label='Measured')
    ax.plot(trial_data['p_des'][:, 0], trial_data['p_des'][:, 1],
            trial_data['p_des'][:, 2], '--', label='Desired')
    ax.set_xlabel('X [m]')
    ax.set_ylabel('Y [m]')
    ax.set_zlabel('Z [m]')
    ax.legend()

    plt.tight_layout()

    # TODO: Phase portraits
    # TODO: EE error over time
    # TODO: Additional comparison plots

# Load and analyze all trials
# trial1_data = load_csv("trial_ctc_nopert.csv")
# trial2_data = load_csv("trial_baseline_nopert.csv")
# trial3_data = load_csv("trial_ctc_pert.csv")
# trial4_data = load_csv("trial_baseline_pert.csv")

# metrics1 = compute_metrics(trial1_data)
# metrics2 = compute_metrics(trial2_data)
# metrics3 = compute_metrics(trial3_data)
# metrics4 = compute_metrics(trial4_data)

# Generate comparison table
# create_comparison_table(metrics1, metrics2, metrics3, metrics4)
```

12 Report Requirements

12.1 Report Structure

Your report must include a title page containing the project title, your name and student ID, date, and course information. Include an abstract of 150-250 words providing a brief overview of objective, methods, and key findings.

The introduction section should cover background on task-space control and IK, motivation for comparing CTC versus PD/PID, and the importance of robustness evaluation. The task definition section must include a table of waypoints with coordinates, justification for the chosen task, and description of the trajectory generation method.

Include a dedicated section titled "Comparison to Reference Code" that explicitly references the file `compare_robust_ctc_vs_pid_lite6_IK_fixed_enhanced.py`. This section must explain how your implementation corresponds to weighted resolved-rate IK, DLS pseudo-inverse, nullspace posture stabilization, reference generation including $\dot{J}$ estimation, and controller comparison methodology. Identify which components were used only in simulation versus on the real robot, and describe any modifications or adaptations made.

The inverse kinematics implementation section should present the mathematical formulation, parameter choices including weights, damping, and gains, and your singularity handling approach. The controller design section must cover the PD/PID controller with gains and anti-windup if applicable, the CTC controller with gains, robust term, and model used, and safety features including saturation and rate limiting.

Document your experimental setup including hardware specifications, control rate and timing, detailed perturbation configuration, and trial execution procedure. Present your results with all required plots organized by trial, the metrics comparison table, and observations for each trial.

The discussion section should compare CTC versus PD/PID performance, analyze the effect of perturbations on each controller, explain observed differences, analyze task-space versus joint-space errors, and address limitations and challenges. Conclude with a summary of key findings and recommendations for real-world deployment.

Include a references section citing the reference code file, course materials, and any additional resources used. Provide appendices containing full controller parameter tables, additional plots if needed, and code snippets for key algorithms.

12.2 Report Formatting Requirements

Submit your report in PDF format with a page limit of 15-25 pages excluding appendices. Use 11pt or 12pt font in a readable typeface with 1.5 or double line spacing and margins of at least 1 inch. All figures must be high quality with readable labels and captions. Equations must be properly formatted and numbered. All plots must have axis labels with units, and all tables must have captions.

13 Submission Requirements

Critical Requirement

Incomplete submissions will not be graded. Verify all components before submitting.

13.1 Required Files

Submit a single compressed archive (.zip or .tar.gz) containing the following components. Include your report as a PDF file named `report_lastname_firstname.pdf`. Provide all source code including all Python scripts, ROS2 launch files, configuration files, and a README with

setup and execution instructions. Include data logs containing ROS bags for all four trials, exported CSV files for all four trials, and metadata files with parameters. Finally, include analysis results containing your Jupyter notebook or Python script for analysis, generated plots as separate files, and a metrics summary in CSV or JSON format.

13.2 Directory Structure

Organize your submission with a root directory named `submission_lastname_firstname` containing a `report` subdirectory with your PDF report, a `code` subdirectory with your README, IK generator, controllers, trial runner, and launch files, a `data` subdirectory with four trial folders each containing rosbag, CSV data, and metadata files, and an `analysis` subdirectory containing your analysis notebook or script, a plots folder with all generated figures, and the metrics summary file.

13.3 Submission Checklist

Before submitting, verify:

- ☐ Report includes all required sections
- ☐ Report explicitly mentions reference code file
- ☐ Task definition meets all requirements (8 waypoints, 2 Z levels, etc.)
- ☐ All four trials completed and logged
- ☐ All required metrics computed
- ☐ All required plots generated
- ☐ Comparison table included in report
- ☐ Code is well-commented and runnable
- ☐ README includes setup instructions
- ☐ All data files are included
- ☐ File naming follows conventions
- ☐ Archive extracts without errors
- ☐ Total file size is reasonable (¡ 500 MB)

14 GRADING RUBRIC

Critical Requirement

Total Points: 120/100

14.1 Task Definition and Implementation (15 points)

Criterion	Points
Defined 8+ distinct waypoints with coordinates documented	3
Two distinct Z levels with ≥ 0.08 m separation	2
Dwell time ≥ 1.0 sec at each waypoint	2
Returns to home configuration	2
Smooth trajectory generation (no velocity jumps)	3
Clear documentation of waypoint table in report	3
Subtotal	**15**

14.2 Inverse Kinematics Implementation (20 points)

Criterion	Points
Weighted resolved-rate IK with $W = \mathrm{diag}(1, 1, w_z)$, $w_z > 1$	5
DLS pseudo-inverse correctly implemented	5
Nullspace posture stabilization included	4
Consistent generation of q_{des}, $\dot{q}_{des}$, $\ddot{q}_{des}$	4
Lightweight $\dot{J}$ estimation (as in reference code)	2
Subtotal	**20**

14.3 Controller Implementation (20 points)

Criterion	Points
PD/PID Baseline (10 points)	
Correct PD or PID implementation	4
Anti-windup if PID (or N/A if PD)	2
Actuator saturation handling	2
Appropriate gain tuning	2
CTC Controller (10 points)	
Correct CTC formulation with dynamics	4
Robust/sliding term (optional but recommended)	2
Actuator saturation handling	2
Appropriate gain tuning	2
Subtotal	**20**

14.4 Perturbation Testing (10 points)

Criterion	Points
Perturbations implemented and documented	4
Identical perturbation config across controller runs	3
Perturbation parameters logged	2
Clear enable/disable timing	1
Subtotal	**10**

14.5 Experimental Execution (10 points)

Criterion	Points
All four required trials completed	4
Same initial configuration for all trials	2
Same IK references used for both controllers	2
Trials executed safely without crashes	2
Subtotal	**10**

14.6 Data Logging (8 points)

Criterion	Points
All required joint-space data logged (q, $\dot{q}$, refs)	2
All required task-space data logged (p, p_{des})	2
Controller outputs logged	1
Perturbation signals/flags logged	1
Data exported to CSV format	1
Proper file naming and organization	1
Subtotal	**8**

14.7 Metrics and Analysis (12 points)

Criterion	Points
Joint RMSE computed for all joints and trials	2
Joint max error computed for all joints and trials	1
Dwell-window mean error computed	2
EE error norm RMSE computed	2
EE max error computed	1
Waypoint success rate computed with justified threshold	2
Comparison table included and complete	2
Subtotal	**12**

14.8 Required Plots (10 points)

Criterion	Points
Joint tracking plots for all 6 joints (all trials)	2
Task-space component plots (X, Y, Z)	2
3D path visualization	1
End-effector error over time	1
Phase portraits for ≥ 3 joints	2
All plots have proper labels, units, legends	1
Plots are clear, readable, and well-organized	1
Subtotal	**10**

14.9 Report Quality (10 points)

Criterion	Points
All required sections present and complete	2
Dedicated section comparing to reference code	2
Explicit mention of reference code filename	1
Clear explanation of CTC vs PD/PID differences	2
Discussion of perturbation effects	1
Professional formatting and writing quality	1
Proper citations and references	1
Subtotal	**10**

14.10 Code Quality and Documentation (5 points)

Criterion	Points
Code is well-commented and readable from a repository in Github	2
README with clear setup instructions	1
Code follows structure template in the PEP8 standard	1
Code is runnable (or detailed explanation if hardware-specific)	1
Subtotal	**5**

14.11 Grading Summary Table

Category	Points	Weight
Task Definition and Implementation	15	15%
Inverse Kinematics Implementation	20	20%
Controller Implementation	20	20%
Perturbation Testing	10	10%
Experimental Execution	10	10%
Data Logging	8	8%
Metrics and Analysis	12	12%
Required Plots	10	10%
Report Quality	10	10%
Code Quality and Documentation	5	5%
TOTAL	**120**	**120%**
Evaluated over	**100**	**100%**

Note: The raw score is out of 120 points but will be normalized to 100 for final grading.

15 Academic Integrity Statement

Critical Requirement

All submitted work must be your own.

You may use the provided reference code as a foundation and discuss concepts with classmates. However, you may not copy code from other students or submit someone else's experimental data. All external sources must be properly cited. Violations will result in a grade of zero and referral to the academic conduct office.

16 Support and Resources

16.1 Office Hours

Office hours are available for questions on IK implementation, controller tuning, data analysis, and report writing.

16.2 Online Resources

Access the reference code repository. Additional documentation is available at ROS2 documentation.

Good luck!